IF2211 Strategi Algoritma

Tugas Besar 2

Dibuat Oleh KelompokRandom3

Muhammad Adam Mirza	18223015
Harfhan Ikhtiar Ahmad Ridzky	18223123

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

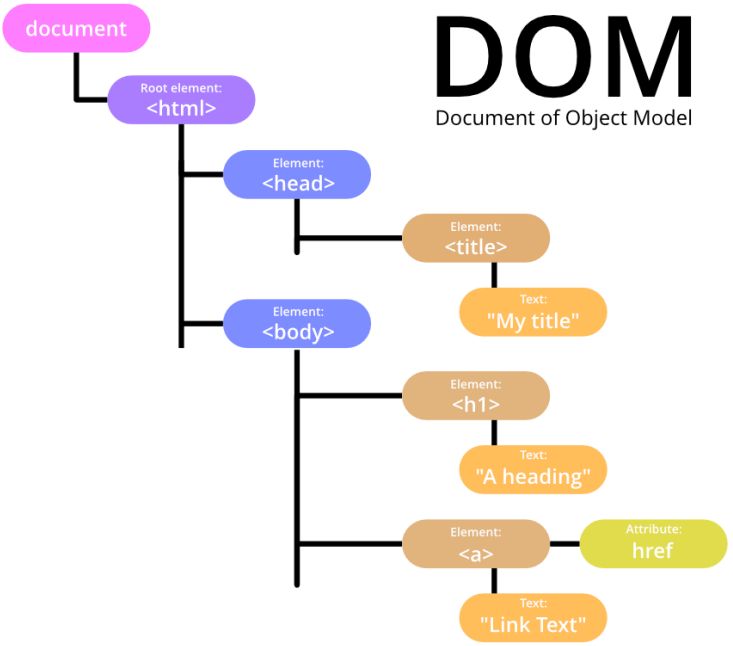
2026

DAFTAR ISI

DAFTAR ISI **2**
DESKRIPSI MASALAH **5**
Komponen-komponen Utama 6
1. Tags 6
2. CSS Selector 6
3. HTML Parsing 7
Spesifikasi Wajib 8
Spesifikasi Bonus yang Diambil 8
LANDASAN TEORI **10**
A. Document Object Model (DOM) 10
B. HTML Parsing 10
C. CSS Selector dan Combinator 10
D. Traversal Graf 11
E. Breadth First Search (BFS) 11
F. Depth First Search (DFS) 11
G. Docker 12
APLIKASI BFS DAN DFS **13**
A. Pemodelan Pohon DOM 13
B. Tokenisasi CSS Selector 13
C. Matching Compound Selector Sederhana 14
D. Evaluasi Combinator dengan Right-to-Left Matching 14
E. Implementasi BFS pada Pohon DOM 15
F. Implementasi DFS pada Pohon DOM 16
G. Analisis Kompleksitas Waktu 17
Tokenisasi Selector 17
Pencocokan matchSimple 17
Pencocokan evaluateChain 17
Kompleksitas BFS dan DFS Secara Keseluruhan 17
H. Analisis Kompleksitas Ruang 18
I. Perbandingan BFS vs DFS pada Konteks DOM 18
IMPLEMENTASI DAN PENGUJIAN **19**
A. Spesifikasi Teknis Program 19
Arsitektur 19
Struktur Data 19
Fungsi Utama 20
Bahasa, Pustaka, dan Tool 21
B. Tata Cara Penggunaan Program 21
C. Pengujian dan Analisis 21

Pengujian 1 BFS dengan Selector Class Sederhana (Input URL).....22
Pengujian 2 DFS dengan Combinator > pada Raw HTML..... 23
Pengujian 3 BFS dengan Combinator Majemuk.....24
D. Catatan Implementasi.....26
BAB 5.....27
KESIMPULAN DAN SARAN..... 27
A. Kesimpulan.....27
B. Saran.....28
Untuk Pengembangan Aplikasi.....28
Untuk Perancang Tugas.....28
C. Refleksi.....28
LAMPIRAN.....29
A. Repository GitHub..... 29
B. Tabel Pemeriksaan.....29
DAFTAR PUSTAKA.....30
PERNYATAAN ORISINALITAS.....31

DESKRIPSI MASALAH



Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk tree yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram.

Struktur sebuah file HTML pada umumnya meliputi elemen root <html> yang berisi dua bagian utama, yaitu <head> dan <body>. Bagian <head> berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan <body> berisi konten utama halaman seperti teks, gambar, tombol, serta elemen HTML lain yang ditampilkan kepada pengguna.

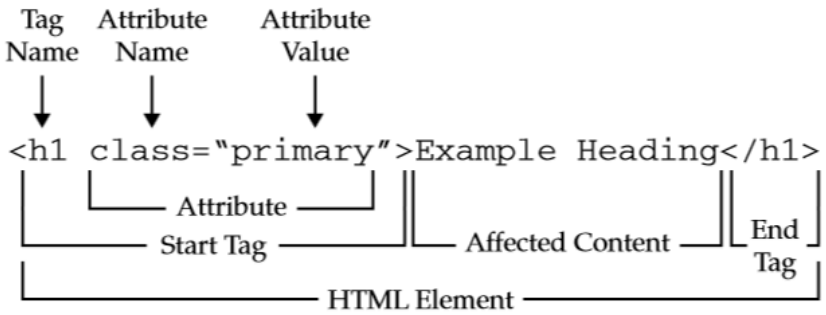
Sebuah website menampilkan DOM melalui file Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ini ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector.

Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

Komponen-komponen Utama

1. Tags

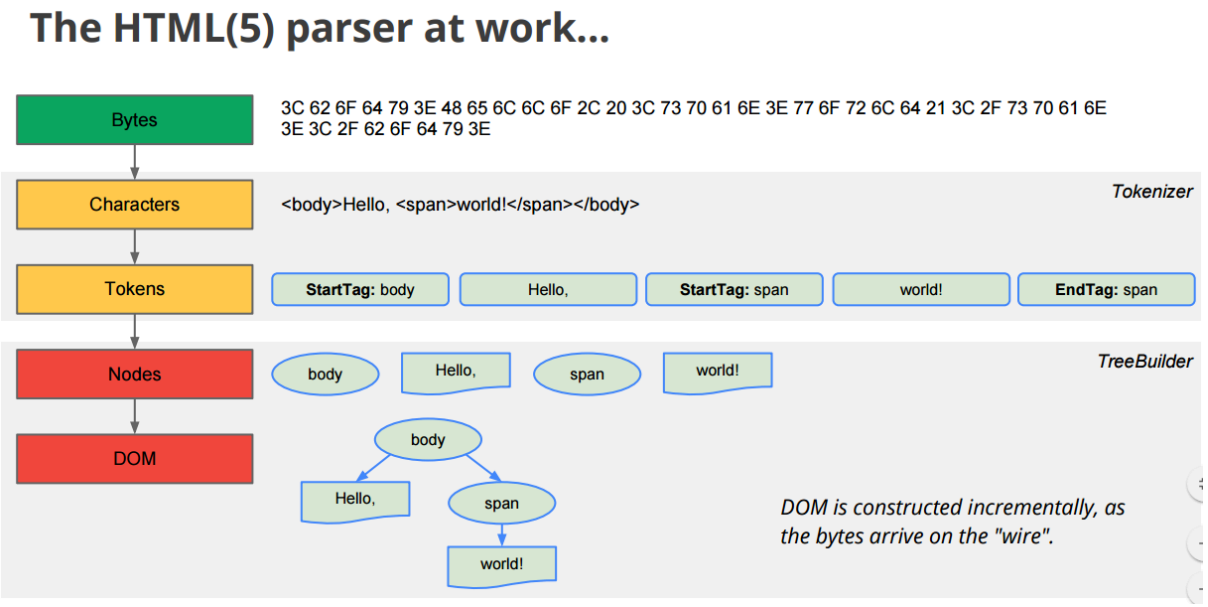
A graphical overview of the HTML markup syntax shown so far is presented here:



Tag pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web. Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (< >). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring (/) sebelum nama tag. Terdapat beberapa jenis tag HTML yang bersifat self-closing.

Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai node pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti parent, child, dan sibling. Struktur inilah yang memungkinkan dokumen HTML diproses sebagai sebuah pohon ketika dilakukan penelusuran menggunakan algoritma seperti BFS atau DFS.

2. CSS Selector



CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Dengan selector, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak. Beberapa selector dasar yang umum digunakan adalah sebagai berikut:

- Tag Selector: memilih elemen berdasarkan nama tag HTML, contoh p memilih semua elemen <p>.
- Class Selector: memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (.), contoh .box.
- ID Selector: memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (#), contoh #header.
- Universal Selector: memilih semua elemen HTML, ditulis sebagai *.

Combinator merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri atas:

- Child Combinator (>) memilih elemen yang merupakan anak langsung.
- Descendant Combinator (spasi) memilih elemen yang merupakan turunan pada kedalaman berapa pun.
- Adjacent Sibling Combinator (+) memilih elemen saudara yang tepat berada setelah elemen sebelumnya.
- General Sibling Combinator (~) memilih semua saudara yang berada setelah elemen tertentu.

3. HTML Parsing

HTML Parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, parser memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag penutup, kemudian menyusunnya ke dalam struktur hierarki.

Setiap tag HTML direpresentasikan sebagai node pada pohon. Tag yang berada di dalam tag lain akan menjadi child node, sedangkan tag yang membungkusnya menjadi parent node. Node paling atas pada struktur ini disebut root, yang umumnya merupakan elemen <html>. Dari node root tersebut, elemen-elemen lain seperti <head> dan <body> akan menjadi cabang yang membentuk struktur pohon DOM.

Hasil dari proses parsing adalah DOM Tree, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma BFS atau DFS.

Spesifikasi Wajib

- Membangun aplikasi berbasis web yang terdiri atas frontend bebas dan backend dalam bahasa Go, C#, Rust, atau TypeScript yang mengimplementasikan algoritma BFS dan DFS untuk melakukan penelusuran CSS pada pohon DOM.
- Aplikasi dapat melakukan scraping HTML dari sebuah website melalui URL yang diberikan oleh pengguna.
- Aplikasi menerima input berupa URL website atau teks HTML, pilihan algoritma traversal (BFS/DFS), CSS selector, serta jumlah hasil yang ingin ditampilkan (Top N kemunculan atau semua kemunculan).
- HTML yang diperoleh diparsing menjadi struktur data pohon (DOM Tree).
- Aplikasi menampilkan visualisasi struktur pohon DOM beserta kedalaman maksimum tree.
- Aplikasi menampilkan highlight pada jalur yang di-traversal oleh algoritma pencarian untuk menandakan elemen-elemen yang terpengaruh.
- Aplikasi menampilkan waktu pencarian serta banyaknya node yang dikunjungi.
- Aplikasi menyimpan traversal log berisi informasi tahapan penelusuran yang dilakukan.

Spesifikasi Bonus yang Diambil

Pada Tugas Besar ini, bonus yang kami kerjakan adalah:

- Docker aplikasi dijalankan menggunakan Docker baik untuk frontend maupun backend, diatur melalui docker-compose.
- Implementasi animasi pada penelusuran pohon untuk membantu dalam memahami pendekatan BFS dan DFS

LANDASAN TEORI

A. Document Object Model (DOM)

Document Object Model (DOM) adalah antarmuka pemrograman lintas platform dan bahasa yang memodelkan dokumen HTML (atau XML) sebagai struktur pohon bercabang, di mana setiap node merepresentasikan bagian dari dokumen. Node pada pohon DOM dapat berupa elemen (tag), atribut, teks, komentar, atau node dokumen itu sendiri.

Pohon DOM berakar pada node dokumen yang kemudian memiliki anak berupa elemen <html>. Elemen <html> membawahi dua anak utama, yaitu <head> yang memuat metadata dan <body> yang memuat konten yang dirender. Hubungan antar-node pada DOM mencakup parent (induk), child (anak), serta sibling (saudara), yang seluruhnya dimanfaatkan dalam penelusuran selector CSS.

Dalam konteks Tugas Besar ini, pohon DOM menjadi graf terarah tanpa siklus (acyclic) dengan derajat percabangan bervariasi. Karena DOM adalah pohon, BFS dan DFS dapat diterapkan langsung tanpa perlu struktur penanda node yang sudah dikunjungi berbeda halnya apabila struktur data memiliki kemungkinan siklus.

B. HTML Parsing

HTML Parsing adalah proses menerjemahkan teks mentah dokumen HTML menjadi struktur data pohon. Proses ini umumnya terbagi atas dua tahap besar: tokenisasi yang mengidentifikasi tag, atribut, dan teks, serta tree construction yang menyusun token ke dalam pohon DOM. Parser yang baik wajib bersifat toleran (error-tolerant) terhadap HTML yang tidak valid, sebagaimana lazim ditemukan di situs nyata.

Pada aplikasi ini, kami menggunakan pustaka standar golang.org/x/net/html, yang merupakan parser HTML5 resmi yang dirawat oleh tim Go. Parser ini mengembalikan node bertipe html.Node yang kemudian kami transformasi menjadi struktur Node internal agar lebih ringan dan mudah ditelusuri.

C. CSS Selector dan Combinator

CSS Selector adalah ekspresi yang mengidentifikasi kumpulan elemen pada pohon DOM. Selector sederhana terdiri atas tag, id (#), class (.), dan atribut. Agar selector dapat mengekspresikan hubungan antar elemen, digunakan combinator:

- Descendant combinator (spasi) A B mencocokkan elemen B yang merupakan turunan dari A pada kedalaman berapa pun.
- Child combinator (>) A > B mencocokkan B yang merupakan anak langsung dari A.

- Adjacent sibling combinator (+) A + B mencocokkan B yang merupakan saudara persis setelah A.
- General sibling combinator (~) A ~ B mencocokkan B yang merupakan saudara setelah A (tidak harus persis setelahnya).

Pencocokan selector dengan combinator umumnya diimplementasikan secara right-to-left, yaitu dimulai dari compound selector paling kanan (elemen yang benar-benar hendak dicocokkan) kemudian memvalidasi syarat ke arah kiri melalui rantai parent atau sibling. Pendekatan ini jauh lebih efisien dibanding left-to-right karena mempersempit ruang pencarian dengan cepat.

D. Traversal Graf

Algoritma traversal graf adalah algoritma yang mengunjungi simpul-simpul di dalam graf terhubung dengan cara sistematis untuk mencari solusi persoalan. Metode ini dapat melakukan pencarian solusi dengan atau tanpa informasi tambahan. Dua tipe traversal graf tanpa informasi tambahan yang paling umum adalah Breadth First Search (BFS) dan Depth First Search (DFS).

Dalam konteks pohon DOM, kedua algoritma dijalankan berangkat dari simpul root dan mengunjungi seluruh elemen pada pohon. Perbedaan utamanya terletak pada urutan kunjungan, yang pada akhirnya berpengaruh pada urutan hasil pencarian ketika pengguna membatasi jumlah hasil dengan Top N.

E. Breadth First Search (BFS)

Misal ditentukan traversal graf dimulai dari simpul v. Algoritma pencarian melebar atau BFS dimulai dengan mengunjungi simpul v, kemudian mengunjungi semua simpul yang bertetangga dengan v terlebih dahulu. Selanjutnya, simpul yang belum dikunjungi dan bertetangga dengan simpul-simpul yang telah dilewati akan dikunjungi. Proses ini berlanjut hingga tidak ada lagi simpul yang belum dikunjungi.

BFS lazimnya diimplementasikan dengan struktur data antrean (queue) yang bersifat First-In-First-Out. Setiap kali suatu simpul diproses, seluruh anaknya dimasukkan ke ujung antrean. Pada pohon DOM, hasil kunjungan BFS dari root akan menelusuri level-per-level: elemen <html>, lalu <head> dan <body>, lalu seluruh anak <head> dan <body>, dan seterusnya.

F. Depth First Search (DFS)

Algoritma pencarian mendalam atau DFS dimulai dengan mengunjungi simpul v, kemudian mengunjungi simpul w yang bertetangga dengan v. Ulangi DFS mulai dari simpul

w. Ketika mencapai simpul u sedemikian sehingga semua simpul yang bertetangga dengannya telah dikunjungi, algoritma melakukan backtrack ke simpul terakhir yang dikunjungi dan masih memiliki simpul tetangga yang belum dikunjungi. Pencarian berakhir bila tidak ada lagi simpul yang belum dikunjungi yang dapat dicapai dari simpul yang telah dikunjungi.

DFS umumnya diimplementasikan secara rekursif memanfaatkan stack memori dari pemanggilan fungsi, atau secara iteratif menggunakan stack eksplisit. Pada pendekatan iteratif dengan stack eksplisit, anak-anak node harus di-push dalam urutan terbalik (reverse order, dari kanan ke kiri) agar yang keluar dari stack pertama kali adalah anak paling kiri. Cara ini memastikan urutan kunjungan DFS yang natural, yaitu mengikuti urutan anak dari kiri ke kanan.

G. Docker

Docker adalah platform virtualisasi berbasis kontainer yang memungkinkan pengembang untuk mengemas aplikasi beserta seluruh dependensinya (kode, runtime, pustaka sistem, dan pengaturan) ke dalam unit standar yang disebut kontainer. Kontainer bersifat ringan, portabel, dan konsisten, sehingga aplikasi dapat berjalan dengan cara yang sama di lingkungan yang berbeda dari laptop pengembang hingga server produksi tanpa khawatir terhadap perbedaan sistem operasi atau konfigurasi.

Docker Compose adalah alat untuk mendefinisikan dan menjalankan aplikasi multi-kontainer. Dengan satu file docker-compose.yml, seluruh service (backend, frontend, database, dan lain-lain) dapat dideskripsikan secara deklaratif, termasuk build context, port mapping, dependencies antar service, dan restart policy. Aplikasi kami memanfaatkan Docker Compose untuk mengorkestrasikan dua service, yaitu backend Go dan frontend Nginx.

APLIKASI BFS DAN DFS

Bab ini memaparkan analisis pemecahan masalah berdasarkan arsitektur aplikasi yang kami bangun. Pembahasan dimulai dari pemodelan pohon DOM, kemudian masuk ke strategi matching CSS selector, dan diakhiri dengan analisis kompleksitas BFS serta DFS pada konteks pohon DOM.

A. Pemodelan Pohon DOM

Langkah pertama dalam menyelesaikan permasalahan ini adalah mengubah dokumen HTML baik dari hasil scraping URL maupun input teks mentah menjadi struktur data pohon yang dapat ditelusuri oleh algoritma. Dalam implementasi, pohon DOM dimodelkan sebagai struct Node bertumpuk yang memiliki field Tag, Attributes (map string ke string), Children (slice Node), Parent (pointer ke parent), dan IsText (penanda apakah node adalah simpul teks).

Kami memanfaatkan pustaka standar golang.org/x/net/html untuk mem-parsing HTML sehingga kami tidak perlu menulis tokenizer sendiri. Namun, struktur html.Node dari pustaka tersebut cukup kompleks karena juga mengandung comment node, doctype node, dan atribut linked list. Oleh karena itu, kami melakukan transformasi walking ke struktur Node internal yang lebih ringkas dan langsung siap ditelusuri.

Key insight pada tahap ini adalah filter pada proses walking: kami membuang node dengan tag script, style, dan noscript karena isi elemen-elemen tersebut adalah kode eksekusi atau deklarasi gaya, bukan elemen dokumen yang relevan bagi CSS Selector pengguna. Kami juga membuang text node yang hanya berisi whitespace karena hal tersebut hanya menambah noise pada visualisasi pohon tanpa memberikan informasi berarti.

Field Parent ditetapkan pada saat walking agar kelak dapat dipakai oleh matcher CSS selector ketika mengevaluasi combinator seperti child (>) atau descendant (spasi). Tanpa pointer parent, matcher harus menelusuri ulang pohon dari root untuk memverifikasi hubungan parent-child, yang akan sangat tidak efisien.

B. Tokenisasi CSS Selector

Setelah pohon DOM tersedia, tantangan berikutnya adalah memecah string CSS selector menjadi daftar token yang dapat dievaluasi. Selector yang diterima pengguna dapat berupa selector sederhana seperti div.container, hingga selector kompleks yang mengombinasikan multiple combinator seperti header nav > ul li.active + li.

Kami mengimplementasikan fungsi parseSelectors yang melakukan dua tahap kerja. Tahap pertama adalah normalisasi whitespace di sekitar combinator: spasi yang berdekatan dengan karakter >, +, atau ~ dibuang agar tidak tertukar dengan descendant combinator. Tanpa normalisasi ini, selector div > p akan memiliki tiga whitespace token yang ambigu.

Tahap kedua adalah tokenisasi karakter-per-karakter. Fungsi ini membangun token buffer dan mengeluarkannya ketika menemui karakter pemisah: spasi (descendant), >, +, atau ~. Hasilnya adalah urutan token yang berselang-seling antara compound selector dan combinator, misalnya selector header nav > ul akan menjadi token [header, ' ', nav, >, ul]. Setelah memiliki daftar token ini, proses pencocokan menjadi lebih sistematis karena matcher cukup berjalan pada indeks tokens tanpa perlu melakukan substring parsing berulang kali.

C. Matching Compound Selector Sederhana

Setiap compound selector (misalnya div#main.active) terdiri atas kombinasi tag, id, dan class. Fungsi matchSimple bertugas menguji apakah satu node memenuhi compound selector tersebut. Pendekatan yang kami ambil adalah memecah compound menjadi tiga komponen melalui state machine sederhana:

- Mode 0 sedang membaca nama tag (sebelum menemui # atau .).
- Mode 1 sedang membaca nilai id (setelah #).
- Mode 2 sedang membaca nama class (setelah .).

Saat menemui karakter # atau ., state machine menyimpan buffer saat ini ke komponen yang tepat dan berpindah mode. Pendekatan ini menangani compound selector dengan multiple class seperti .btn.btn-primary.active secara natural.

Pencocokan tag dilakukan dengan perbandingan string langsung. Pencocokan id dilakukan terhadap atribut id node. Pencocokan class dilakukan terhadap token-token pada atribut class yang dipisahkan oleh whitespace sebuah node dianggap cocok jika seluruh class yang diminta selector ada pada class node tersebut.

Selector universal * serta compound selector kosong ditangani sebagai kasus khusus dan selalu menghasilkan kecocokan untuk setiap element node non-teks.

D. Evaluasi Combinator dengan Right-to-Left Matching

Bagian ini merupakan inti dari matcher CSS selector dan merupakan key insight utama dalam implementasi. Ketika selector mengandung combinator, matcher tidak dapat sekadar mengecek satu compound selector pada satu node ia harus memvalidasi rantai kondisi yang mengikat beberapa node melalui hubungan parent, descendant, atau sibling.

Kami memilih strategi right-to-left matching karena compound selector paling kanan adalah target akhir (yaitu node yang hendak dikembalikan sebagai hasil), sehingga wajar untuk mulai memverifikasi dari situ. Strategi ini juga konsisten dengan cara kerja mesin CSS pada browser modern seperti Chromium dan Firefox.

Fungsi evaluateChain bekerja rekursif: diberikan node n, daftar token tokens, dan indeks tokenIdx yang menunjuk ke compound selector yang sedang dievaluasi, fungsi ini menguji apakah n memenuhi compound pada tokenIdx. Jika ya, ia mundur ke compound sebelumnya (tokenIdx - 2, karena token di tengah adalah combinator) dan melanjutkan verifikasi pada node yang sesuai dengan hubungan combinator:

- Combinator > (child): verifikasi dilakukan pada n.Parent.
- Combinator spasi (descendant): verifikasi dilakukan pada seluruh leluhur n (loop naik sampai nil), dan cukup salah satu yang valid.
- Combinator + (adjacent sibling): verifikasi dilakukan pada saudara non-teks tepat sebelum n di bawah parent yang sama.
- Combinator ~ (general sibling): verifikasi dilakukan pada seluruh saudara non-teks yang berada sebelum n, dan cukup salah satu yang valid.

Rekursi berhenti ketika tokenIdx telah mencapai compound paling kiri (indeks 0) dan kondisi terpenuhi, yang berarti seluruh rantai selector berhasil di-match. Pendekatan ini secara alami menangani selector dengan sembarang jumlah combinator bersarang.

E. Implementasi BFS pada Pohon DOM

Setelah matcher selector siap, langkah berikutnya adalah implementasi BFS untuk menelusuri pohon DOM dari root dan mengumpulkan seluruh node yang cocok dengan selector. BFS kami menggunakan struktur antrean berbasis slice Go, dengan operasi enqueue di ujung slice dan dequeue di awal slice.

Detail algoritma BFS yang kami implementasikan adalah sebagai berikut:

1. Inisialisasi antrean q dengan root pohon DOM sebagai elemen pertama.
2. Selama antrean tidak kosong, ambil elemen paling depan sebagai curr.

3. Jika curr bukan text node, tambahkan counter node-yang-dikunjungi dan catat deskripsi curr pada traversal log (contoh: div#main.container).
4. Panggil matchComplex(curr, selector); jika cocok, tambahkan ke daftar hasil. Jika jumlah hasil telah mencapai batas Top N yang diminta pengguna, hentikan loop.
5. Apabila loop belum berhenti, tambahkan seluruh anak curr ke ujung antrean, lalu lanjutkan ke iterasi berikutnya.

Karena pohon DOM tidak memiliki siklus, kami tidak perlu struktur visited tambahan. Setiap node akan dikunjungi tepat satu kali, kecuali loop dihentikan lebih awal karena Top N tercapai.

F. Implementasi DFS pada Pohon DOM

DFS kami diimplementasikan secara iteratif memanfaatkan stack eksplisit berbasis slice Go, bukan rekursif. Pilihan ini diambil untuk menghindari risiko stack overflow pada pohon DOM yang sangat dalam, serta untuk memberikan kontrol granular atas urutan kunjungan.

Detail algoritma DFS yang kami implementasikan adalah sebagai berikut:

1. Inisialisasi stack dengan root pohon DOM.
2. Selama stack tidak kosong, pop elemen paling atas sebagai curr.
3. Catat curr pada traversal log dan tingkatkan counter node-yang-dikunjungi (dengan syarat curr bukan text node).
4. Jika curr cocok dengan selector, tambahkan ke daftar hasil. Hentikan loop jika Top N tercapai.
5. Push anak-anak curr ke stack dalam urutan terbalik (dari anak terakhir ke anak pertama).

Key insight pada DFS adalah urutan push anak yang terbalik (reverse order). Jika anak di-push dari kiri ke kanan, maka yang pertama kali di-pop adalah anak paling kanan, sehingga urutan kunjungan DFS akan terbalik dari yang lazim. Dengan push dalam urutan terbalik, anak paling kiri berada di puncak stack dan akan di-pop duluan, menghasilkan urutan kunjungan yang sesuai dengan ekspektasi (kiri-ke-kanan, depth-first).

G. Analisis Kompleksitas Waktu

Misalkan n adalah jumlah total node pada pohon DOM, d adalah kedalaman maksimum pohon, dan s adalah panjang compound selector pada CSS selector. Analisis kompleksitas berikut berlaku pada skenario terburuk.

Tokenisasi Selector

Fungsi parseSelectors berjalan satu kali per query dengan kompleksitas $O(|selector|)$. Biaya ini dapat diabaikan karena selector biasanya sangat pendek dibandingkan dengan pohon DOM.

Pencocokan matchSimple

Kompleksitas matchSimple adalah $O(s + k)$ di mana s adalah panjang compound selector dan k adalah jumlah class pada atribut class node. Pada praktiknya keduanya kecil dan dapat dianggap konstan $O(1)$ dalam analisis agregat.

Pencocokan evaluateChain

Kompleksitas evaluateChain bergantung pada tipe combinator pada rantai. Combinator > memeriksa tepat satu parent *(O(1) per langkah)*. Combinator spasi dapat memeriksa hingga seluruh leluhur *(O(d) per langkah)*. Combinator + memeriksa tepat satu saudara *(O(1))*. Combinator ~ memeriksa seluruh saudara sebelumnya *(O(cabang parent))*. Pada selector dengan c combinator, kompleksitas worst-case evaluateChain adalah $O(c \cdot d)$ ketika seluruh combinator adalah descendant.

Kompleksitas BFS dan DFS Secara Keseluruhan

Pada kasus pengguna meminta seluruh kemunculan $Top\ N = \frac{0}{\infty}$, BFS dan DFS keduanya mengunjungi seluruh n node. Untuk setiap node, matcher dijalankan dengan biaya $O(c \cdot d)$ pada worst-case. Kompleksitas total keduanya menjadi $O(n \cdot c \cdot d)$. Untuk selector sederhana tanpa combinator (c = 1), kompleksitas turun menjadi $O(n)$.

Jika pengguna meminta Top N dengan N lebih kecil dari jumlah total kecocokan, BFS memiliki keuntungan berupa jaminan bahwa hasil yang dikembalikan adalah N kemunculan pertama secara level-per-level (dekat dengan root), sementara DFS mengembalikan N kemunculan pertama secara depth-first (lorong kiri pohon). Keduanya menghentikan traversal lebih awal ketika Top N tercapai, sehingga biaya aktualnya lebih murah daripada batas atas teoretis.

H. Analisis Kompleksitas Ruang

Kompleksitas ruang BFS didominasi oleh antrean. Pada kasus terburuk, antrean dapat berisi seluruh node pada level terlebar dari pohon DOM, yang pada pohon seimbang berbanding n/d dan pada pohon ekstrem berbanding n. Dengan demikian, ruang antrean BFS adalah *O(n)* pada worst-case dan $O(\frac{n}{d})$ pada pohon seimbang.

Kompleksitas ruang DFS didominasi oleh stack. Stack pada DFS iteratif dapat berisi hingga *O(b · d)* elemen, di mana b adalah faktor percabangan rata-rata. Pada pohon DOM yang tidak terlalu lebar, stack DFS cenderung lebih hemat memori dibandingkan antrean BFS, terutama pada pohon yang sangat dalam dan kurus.

Selain itu, traversal log menambah ruang *O(n)* karena setiap node non-teks yang dikunjungi ditambahkan ke log. Daftar hasil juga menambah ruang O(jumlah kecocokan), yang dibatasi oleh Top N apabila diaktifkan.

I. Perbandingan BFS vs DFS pada Konteks DOM

Meskipun BFS dan DFS memiliki kompleksitas waktu worst-case yang sama *(O(n · c · d))*, perilaku keduanya pada kasus praktis berbeda cukup signifikan:

- BFS menghasilkan urutan hasil yang "breadth-oriented": elemen yang lebih dekat dengan root akan ditemukan lebih dulu. Cocok bila pengguna mencari landmark elemen seperti <header>, <nav>, atau <main> pada level atas dokumen.
- DFS menghasilkan urutan hasil yang "depth-oriented": elemen pada subtree kiri ditelusuri sampai tuntas sebelum pindah ke subtree kanan. Cocok bila pengguna ingin menelusuri struktur bersarang dalam, misalnya mencari seluruh <li> pada menu hirarkis.
- Pada query Top N, BFS memberikan jaminan lokalitas level sementara DFS memberikan jaminan lokalitas subtree. Pilihan antar keduanya bergantung pada preferensi pengguna terhadap hasil yang diinginkan.
- Pada pohon DOM yang sangat dalam, DFS iteratif kami lebih aman dari risiko stack overflow dibandingkan DFS rekursif, dan tetap hemat memori dibandingkan BFS karena stack hanya menyimpan jalur aktif pencarian.

IMPLEMENTASI DAN PENGUJIAN

A. Spesifikasi Teknis Program

Arsitektur

Aplikasi kami dirancang dengan arsitektur client-server dua kontainer yang dikelola oleh Docker Compose. Komunikasi frontend-backend dilakukan melalui reverse proxy Nginx yang meneruskan request dengan prefix /api/ ke service backend.

- Backend: ditulis dalam bahasa Go murni (standard library + golang.org/x/net/html). Backend menyediakan endpoint HTTP POST /api/search yang menerima payload JSON berisi input pengguna dan mengembalikan hasil penelusuran dalam format JSON.
- Frontend: ditulis dengan vanilla HTML, CSS, dan JavaScript tanpa framework tambahan. Frontend di-serve oleh Nginx yang juga bertugas sebagai reverse proxy ke backend.
- Docker Compose: mengorkestrasikan kedua service, mengatur port mapping (frontend pada port 3000), serta dependency backend → frontend.

Struktur Data

Nama	**Deskripsi**
Node	Struct Go yang merepresentasikan satu simpul pada pohon DOM. Berisi field Tag, Attributes (map[string]string), Children ([]*Node), Parent (*Node), dan IsText (bool).
Request	Struct payload input dari frontend. Berisi URL, HtmlContent, Selector, Algo, dan Limit.
Response	Struct output ke frontend. Berisi Results ([]*Node), TraversalLog ([]string), NodeCount, ExecutionTimeMs, MaxDepth, dan RootTree.
Queue (slice)	Struktur FIFO untuk BFS, berbasis slice Go dengan enqueue di belakang dan dequeue di depan.
Stack (slice)	Struktur LIFO untuk DFS, berbasis slice Go dengan push dan pop di belakang slice.

tokens ([]string)	Hasil tokenisasi CSS selector, berisi compound selector dan combinator secara bergantian.

Fungsi Utama

Nama Fungsi	**Deskripsi**
ParseHTML(url)	Melakukan HTTP GET ke URL dan mem-parsing respons HTML menjadi pohon Node.
ParseHTMLText(html)	Mem-parsing teks HTML mentah yang diberikan pengguna menjadi pohon Node.
parseReader(reader)	Fungsi internal yang melakukan walking html.Node menjadi Node kustom. Menyaring tag script/style/noscript dan whitespace text.
parseSelectors(sel)	Tokenisasi CSS selector menjadi slice token berisi compound selector dan combinator.
matchSimple(n, sel)	Mengevaluasi apakah satu node cocok dengan compound selector sederhana (tag, id, class).
evaluateChain(n, tokens, idx)	Fungsi rekursif yang mengevaluasi rantai compound selector secara right-to-left dengan memperhatikan combinator.
matchComplex(n, sel)	Wrapper yang melakukan tokenisasi lalu memanggil evaluateChain.
BFS(root, selector, limit)	Implementasi iteratif BFS dengan queue slice. Mengembalikan daftar hasil, traversal log, dan jumlah node yang dikunjungi.
DFS(root, selector, limit)	Implementasi iteratif DFS dengan stack slice. Anak di-push dalam urutan terbalik agar kiri-ke-kanan.
MaxDepth(n)	Menghitung kedalaman maksimum pohon DOM secara rekursif.

searchHandler(w, r)	HTTP handler pada endpoint /api/search. Melakukan decode JSON, memilih mode scraping (URL/teks), memanggil algoritma, lalu mengembalikan response JSON.
corsMiddleware(next)	Middleware CORS yang menambahkan header Access-Control-Allow-* serta menangani preflight OPTIONS.

Bahasa, Pustaka, dan Tool

- Bahasa backend: Go (modul dideklarasikan di go.mod).
- Pustaka utama: net/http (standar), encoding/json (standar), strings (standar), time (standar), golang.org/x/net/html (parser HTML).
- Bahasa frontend: HTML5, CSS3, JavaScript (ES6+), tanpa framework.
- Web server frontend: Nginx (alpine image) sekaligus reverse proxy ke backend.
- Containerization: Docker dan Docker Compose.

B. Tata Cara Penggunaan Program

Halaman utama terdiri atas satu panel input dan satu area hasil yang muncul setelah submit. Pada panel input, pengguna dapat memilih mode input antara Scrape URL dan Raw HTML Text. Mode Scrape URL akan menampilkan input URL, sementara mode Raw HTML Text akan menampilkan text area untuk menempelkan teks HTML langsung. Selanjutnya pengguna mengisi CSS Selector, memilih algoritma (BFS atau DFS) pada dropdown, dan secara opsional mengisi jumlah hasil pada field Top N Results (kosongkan untuk mendapatkan seluruh kemunculan).

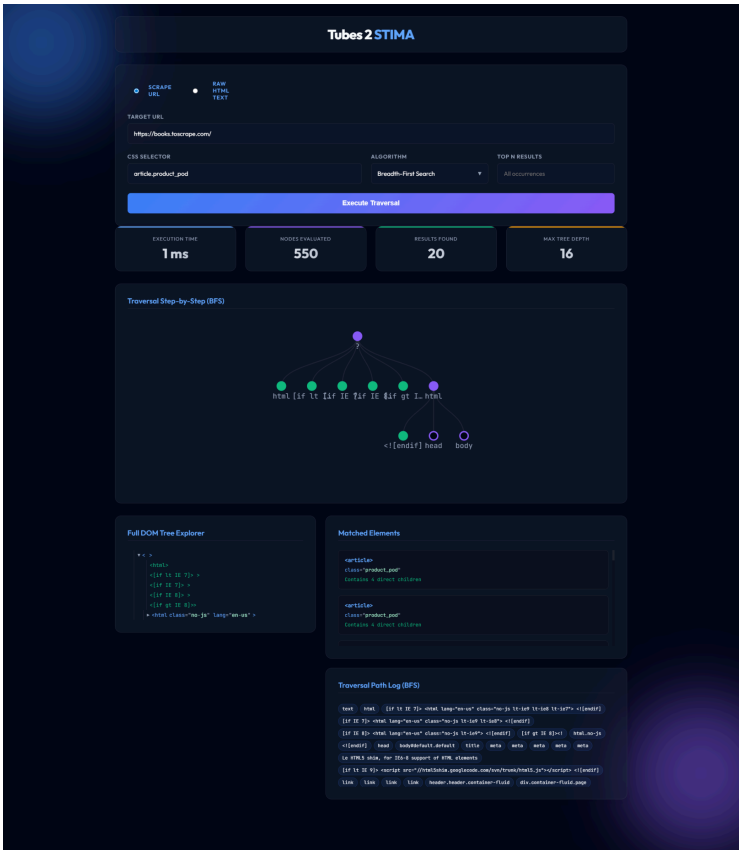
Setelah tombol Execute Traversal ditekan, aplikasi mengirim request ke backend dan menampilkan hasil pada bagian bawah halaman. Hasil terdiri atas empat panel metrik (Execution Time, Nodes Evaluated, Results Found, Max Tree Depth), panel Full DOM Tree Explorer (pohon DOM yang dapat di-collapse/expand melalui elemen <details>), panel Matched Elements (daftar elemen yang cocok dengan selector), dan panel Traversal Path Log yang menampilkan chip animasi untuk setiap node yang dikunjungi.

C. Pengujian dan Analisis

Tiga skenario pengujian berikut disusun untuk memverifikasi fungsionalitas aplikasi pada berbagai tipe input dan kompleksitas selector. Setiap pengujian didokumentasikan dengan screenshot hasil yang dilampirkan pada laporan final.

Pengujian 1 BFS dengan Selector Class Sederhana (Input URL)

- Input mode: Scrape URL
- URL: https://books.toscrape.com/
- CSS Selector: article.product_pod
- Algoritma: BFS
- Jumlah hasil: kosong (seluruh kemunculan)



Pengujian ini memverifikasi bahwa aplikasi dapat melakukan scraping pada URL publik, mem-parsing HTML yang diterima secara utuh, serta mengeksekusi algoritma BFS untuk mencari seluruh elemen katalog buku dengan class `product_pod`. Panel metrik menampilkan waktu eksekusi dalam milidetik, jumlah node yang dikunjungi, jumlah elemen yang cocok, dan kedalaman maksimum pohon DOM. Traversal log pada bagian bawah menunjukkan urutan kunjungan node secara level-per-level yang khas pada metode BFS.

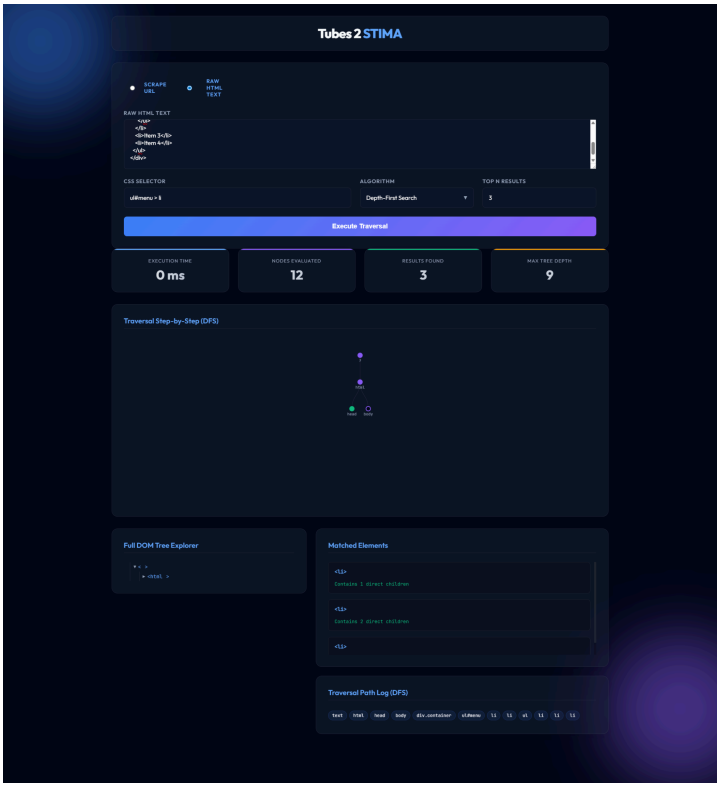
Kunjungan dimulai dari root HTML, menuju head, lalu body, dan secara bertahap turun menelusuri elemen-elemen secara mendatar di dalamnya.

Pengujian 2 DFS dengan Combinator > pada Raw HTML

- Input mode: Raw HTML Text
- HTML:

```
<div class="container">
  <ul id="menu">
    <li>Item 1</li>
    <li>Item 2
      <ul>
        <li>Subitem 2.1</li>
        <li>Subitem 2.2</li>
      </ul>
    </li>
    <li>Item 3</li>
    <li>Item 4</li>
  </ul>
</div>
```

- CSS Selector: ul#menu > li
- Algoritma: DFS
- Jumlah hasil: 3 (Top 3)



Pengujian ini memverifikasi fungsi *combinator child* (>) yang secara spesifik hanya mencocokkan tag `<li>` sebagai anak langsung dari `<ul>` dengan ID `menu`, sehingga tag `<li>` yang berada di dalam `<ul>` bersarang (*subitem*) tidak akan dicocokkan. Selain itu, pengujian ini juga menguji fitur pembatasan jumlah hasil (*Top N*) yang menghentikan proses traversal tepat setelah 3 elemen ditemukan. Urutan log traversal menunjukkan karakteristik algoritma DFS (*Depth-First Search*) yang berfokus menyelam sedalam mungkin ke subtree paling kiri terlebih dahulu, dan fitur *Top N* terbukti bekerja dengan baik karena terlihat dari metrik jumlah *Nodes Evaluated* yang jauh lebih kecil dibandingkan jumlah total node pada pohon secara keseluruhan, sehingga traversal benar-benar berhenti lebih awal setelah elemen ketiga yang valid ditemukan.

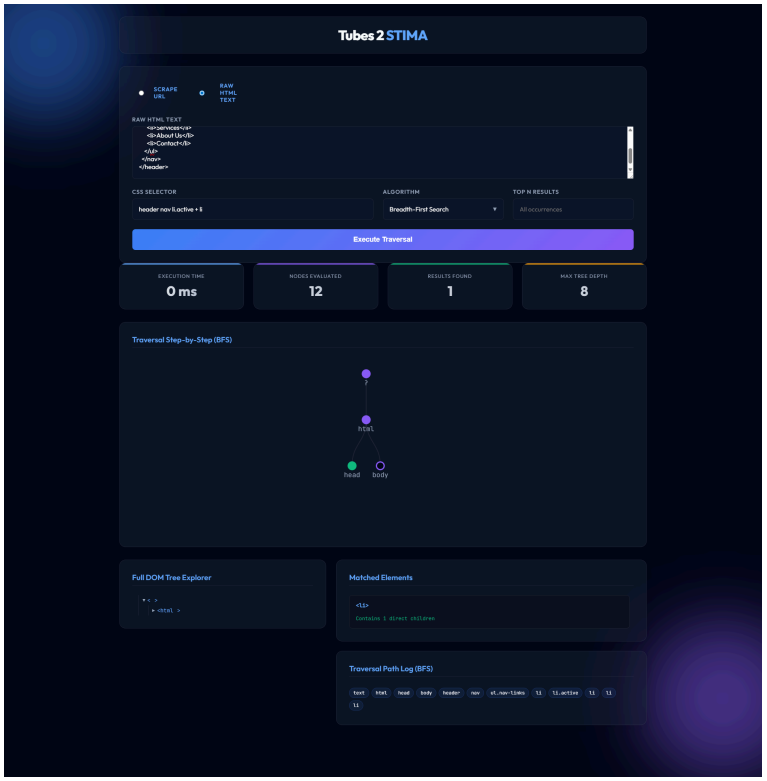
Pengujian 3 BFS dengan Combinator Majemuk

- Input mode: Raw HTML Text
- HTML:

```
<header>
  <nav>
    <ul class="nav-links">
      <li>Home</li>
      <li class="active">Products</li>
      <li>Services</li>
```

```
        <li>About Us</li>
        <li>Contact</li>
      </ul>
    </nav>
  </header>
```

- CSS Selector: header nav li.active + li
- Algoritma: BFS
- Jumlah hasil: kosong (seluruh kemunculan)



Pengujian ini bertujuan memverifikasi keandalan fungsi *matcher* dalam mengevaluasi rantai *combinator* tingkat lanjut yang menggabungkan *descendant* (`header nav li`) dengan *adjacent sibling* (**+**), di mana kandidat hasil yang valid adalah setiap tag `<li>` yang berada tepat setelah `<li>` berkelas `active` dan terletak dalam struktur turunan dari `header` menuju `nav`. Strategi *matching* kanan-ke-kiri pada sistem bekerja secara efisien dengan memulai dari komponen paling kanan (`li`), kemudian melangkah mundur melalui *adjacent sibling* ke elemen saudara sebelumnya yang harus cocok dengan `li.active`, lalu terus dievaluasi ke atas menuju `nav` dan `header`, sehingga jalur evaluasi mundur ini menunjukkan keunggulan pendekatan kanan-ke-kiri dalam memangkas dan mempersempit ruang pencarian secara cepat dibandingkan pendekatan konvensional kiri-ke-kanan.

D. Catatan Implementasi

Terdapat beberapa catatan penting yang kami rasa perlu disampaikan dalam laporan ini:

- Highlight jalur traversal pada aplikasi diwujudkan dalam visual pohon dan bentuk chip animasi pada panel Traversal Path Log. Setiap chip muncul dengan animasi pop-in (scale + fade) secara bertahap menggunakan requestAnimationFrame untuk menjaga responsivitas pada log berukuran besar. Ditambah visual pohon untuk memberikan gambaran yang jelas mengenai urutan kunjungan pada tree yang sudah interaktif.
- Parser kami dengan sengaja membuang node script, style, dan noscript karena konten tag tersebut bukanlah elemen dokumen yang relevan bagi pengguna CSS Selector. Keputusan ini mengurangi noise pada visualisasi dan traversal log. Apabila pengguna benar-benar ingin menelusuri tag tersebut, perilaku filter ini dapat dinonaktifkan dengan mudah pada fungsi parseReader.
- Pada arsitektur Docker Compose, backend tidak meng-expose port-nya ke host hanya frontend yang di-expose pada port 3000. Komunikasi frontend ke backend terjadi di dalam jaringan Docker internal melalui Nginx reverse proxy. Desain ini mengurangi surface area dan memastikan bahwa endpoint API hanya dapat diakses melalui frontend.

KESIMPULAN DAN SARAN

A. Kesimpulan

Dalam Tugas Besar 2 IF2211 Strategi Algoritma ini, kami telah merancang dan mengimplementasikan aplikasi web untuk penelusuran elemen pada struktur Document Object Model berdasarkan CSS Selector dengan memanfaatkan algoritma Breadth First Search dan Depth First Search. Implementasi backend dilakukan dalam bahasa Go dengan pustaka standar net/http dan golang.org/x/net/html, sedangkan frontend dibangun dengan vanilla HTML, CSS, dan JavaScript. Kedua service dijalankan sebagai kontainer terpisah yang diorkestrasikan oleh Docker Compose dengan Nginx sebagai reverse proxy.

Algoritma BFS kami implementasikan dengan antrean berbasis slice Go, menghasilkan urutan kunjungan level-per-level yang cocok untuk menemukan elemen dekat root. Algoritma DFS kami implementasikan secara iteratif dengan stack eksplisit dan teknik push anak dalam urutan terbalik agar urutan kunjungan tetap natural dari kiri ke kanan. Keduanya menghentikan traversal lebih awal ketika batas Top N tercapai, memberikan efisiensi tambahan pada query dengan hasil terbatas.

Key insight paling signifikan pada implementasi kami adalah strategi right-to-left matching pada fungsi evaluateChain untuk mengevaluasi CSS selector yang mengandung combinator. Strategi ini mengikuti pendekatan yang digunakan mesin CSS pada browser modern dan terbukti efisien dalam mempersempit ruang pencarian. Matcher kami mendukung seluruh combinator yang disyaratkan pada spesifikasi tugas (descendant, child, adjacent sibling, general sibling) serta compound selector yang mengombinasikan tag, id, dan class.

Fitur-fitur wajib yang telah kami penuhi meliputi: scraping URL dan input teks HTML mentah, pemilihan algoritma BFS/DFS, input CSS Selector, pilihan Top N atau seluruh kemunculan, visualisasi pohon DOM interaktif yang dapat di-collapse, penanda jalur traversal dalam bentuk chip animasi pada panel log, penampilan waktu eksekusi, jumlah node yang dikunjungi, serta kedalaman maksimum pohon. Untuk bonus, kami berhasil melakukan containerization aplikasi dengan Docker Compose.

B. Saran

Dari pengalaman mengerjakan Tugas Besar ini, kami ingin menyampaikan beberapa saran baik untuk pengembangan lanjutan aplikasi maupun untuk perancang tugas di masa depan.

Untuk Pengembangan Aplikasi

- Implementasi multithreading dengan goroutine dapat dipertimbangkan untuk paralelisasi traversal subtree pada pohon DOM yang sangat besar. Pembagian kerja dapat dilakukan per-child dari root, dengan sinkronisasi daftar hasil menggunakan channel atau mutex.
- Penambahan pseudo-class seperti :first-child, :nth-child(n), dan :not(selector) akan membuat matcher kami lebih kompatibel dengan spesifikasi CSS Selectors Level 4.
- Implementasi fitur bonus lain yang belum sempat kami kerjakan terutama Lowest Common Ancestor dengan Binary Lifting adalah pengembangan yang menarik dan secara konseptual mendukung analisis struktural pohon DOM.

Untuk Perancang Tugas

- Penyediaan contoh HTML test set yang terstandar akan memudahkan seluruh kelompok membandingkan perilaku algoritma secara apple-to-apple, serta mempercepat sesi demo.
- Contoh struktur pohon DOM hasil parsing pada spek akan sangat membantu mahasiswa dalam memverifikasi implementasi parser-nya.

C. Refleksi

Melalui Tugas Besar ini, kami mendapatkan pemahaman yang jauh lebih dalam mengenai algoritma BFS dan DFS melengkapi materi kelas, terutama pada aspek implementasi iteratif dengan struktur data eksplisit dan adaptasi pada struktur pohon khusus seperti DOM. Tantangan terbesar yang kami hadapi adalah mendesain matcher CSS Selector yang menangani combinator secara benar bukan pada algoritma traversal-nya sendiri. Pengerjaan tugas ini memperkuat keyakinan kami bahwa pemilihan struktur data dan pendekatan matching seringkali lebih menentukan kualitas solusi daripada algoritma traversal yang dipilih.

LAMPIRAN

A. Repository GitHub

https://github.com/AdmMRZ/Tubes2_KelompokRandom3

B. Tabel Pemeriksaan

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video		✓
10	[Bonus] Deploy aplikasi		✓
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading		✓
13	[Bonus] Implementasi LCA Binary Lifting		✓
14	[Bonus] Docker (frontend & backend)	✓	

DAFTAR PUSTAKA

Munir, R. (2025). Breadth First Search dan Depth First Search Bagian 1. Bahan Kuliah IF2211 Strategi Algoritma, Institut Teknologi Bandung. Diakses dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/13-BFS-DFS-(2025)-Bagian1.pdf

Munir, R. (2025). Breadth First Search dan Depth First Search Bagian 2. Bahan Kuliah IF2211 Strategi Algoritma, Institut Teknologi Bandung. Diakses dari https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/14-BFS-DFS-(2025)-Bagian2.pdf

Mozilla Developer Network. (2026). CSS Selectors. Diakses dari https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Selectors

Mozilla Developer Network. (2026). Client-Server Overview. Diakses dari https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/First_steps/Client-Server_overview

The Go Authors. (2026). Package html - golang.org/x/net/html. Diakses dari https://pkg.go.dev/golang.org/x/net/html

The Go Authors. (2026). Package net/http. Diakses dari https://pkg.go.dev/net/http

Docker Inc. (2026). Docker Documentation. Diakses dari https://docs.docker.com/

Docker Inc. (2026). Docker Compose Documentation. Diakses dari https://docs.docker.com/compose/

World Wide Web Consortium. (2018). Document Object Model (DOM) Level 1. Diakses dari https://www.w3.org/TR/WD-DOM/

GeeksforGeeks. (2024). What is Document Object in Java DOM? Diakses dari https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/

PERNYATAAN ORISINALITAS

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Muhammad Adam Mirza

18223015

Harfhan Ikhtiar Ahmad Ridzky

18223123